EXPERIMENT NO. 01: YOUR FIRST JAVASCRIPT SCRIPT

Experiment Title: Your First JavaScript Script

Aim: Run a script in the browser and with Node, and declare values with const and let.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: JavaScript is a scripting language used to make web pages interactive. A script can run in a browser or with Node.js. The const keyword declares a binding that cannot be reassigned, while let declares a reassignable block-scoped variable. The typeof operator reports the type of a value.

Algorithm / Procedure:

- Task 1 – Print a message in the browser.
- Task 2 – Declare and display values with const and let.
- Task 3 – Display the types of values using typeof.

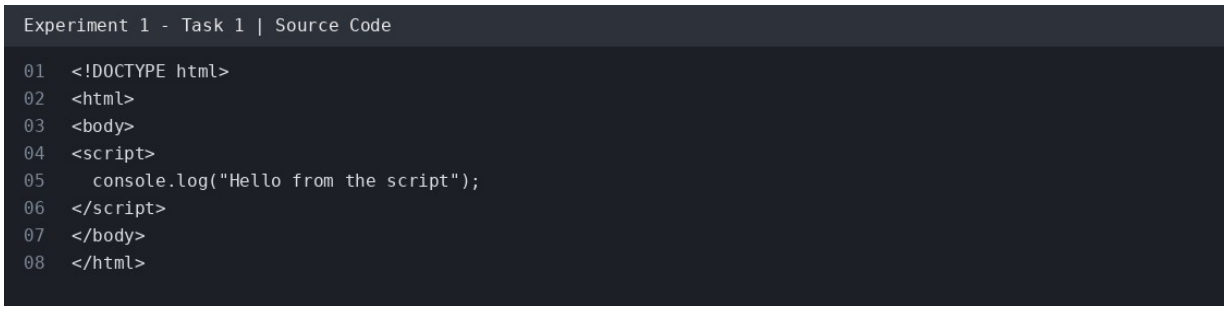
Program / Source Code & Output:

Task 1 – Print a message in the browser

Source Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
  console.log("Hello from the script");
</script>
</body>
</html>
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Declare and display values with const and let

Source Code:

```
const schoolName = "Sample College";
let year = 2026;

console.log(schoolName);
```

```
console.log(year);
```

Source Code Screenshot:

Experiment 1 - Task 2 | Source Code

```
01  const schoolName = "Sample College";
02  let year = 2026;
03
04  console.log(schoolName);
05  console.log(year);
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Sample College
2026
```

Task 3 – Display the types of values using typeof

Source Code:

```
console.log(typeof "hello");
console.log(typeof 7);
console.log(typeof true);
```

Source Code Screenshot:

Experiment 1 - Task 3 | Source Code

```
01  console.log(typeof "hello");
02  console.log(typeof 7);
03  console.log(typeof true);
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
string
number
boolean
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Your First JavaScript Script” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is JavaScript? — A scripting language used to create dynamic web content.
- What is the difference between const and let? — const cannot be reassigned; let can be reassigned.
- What does typeof return? — It returns a string describing the type of a value.

EXPERIMENT NO. 02: OPERATORS, TYPE CONVERSION AND A CALCULATOR

Experiment Title: Operators, Type Conversion and a Calculator

Aim: Use arithmetic operators, convert between text and numbers, and use if and else if to choose an operation.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: Arithmetic operators perform calculations such as addition, subtraction, multiplication and division. Number() converts suitable text to a number, while String() converts a value to text. Conditional statements select code according to a condition.

Algorithm / Procedure:

- Task 1 – Perform arithmetic operations.
- Task 2 – Convert between text and numbers.
- Task 3 – Select an operation with if and else if.

Program / Source Code & Output:

Task 1 – Perform arithmetic operations

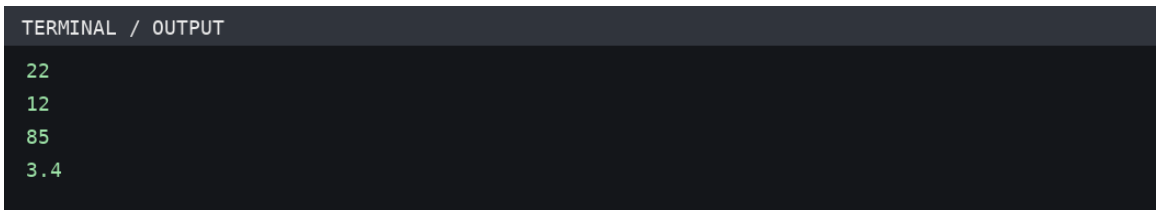
Source Code:

```
const a = 17;
const b = 5;
console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Convert between text and numbers

Source Code:

```
const textNumber = "12";
console.log(Number(textNumber) + 5);

const numberValue = 7;
console.log(String(numberValue) + "3");
```

Source Code Screenshot:

Experiment 2 - Task 2 | Source Code

```
const textNumber = "12";
console.log(Number(textNumber) + 5);

const numberValue = 7;
console.log(String(numberValue) + "3");
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
17
73
```

Task 3 – Select an operation with if and else if

Source Code:

```
const a = 8;
const b = 4;
let operator = "+";

if (operator === "+") {
  console.log(a + b);
} else if (operator === "-") {
  console.log(a - b);
}

operator = "-";
if (operator === "+") {
  console.log(a + b);
} else if (operator === "-") {
  console.log(a - b);
}
```

Source Code Screenshot:

Experiment 2 - Task 3 | Source Code

```
const a = 8;
const b = 4;
let operator = "+";

if (operator === "+") {
  console.log(a + b);
} else if (operator === "-") {
  console.log(a - b);
}

operator = "-";
if (operator === "+") {
  console.log(a + b);
} else if (operator === "-") {
  console.log(a - b);
}
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
12
4
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Operators, Type Conversion and a Calculator” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- Name four arithmetic operators. — +, -, * and /.
- What does Number() do? — It converts a value to a number when possible.
- What is an else-if ladder? — It checks multiple conditions in sequence.

EXPERIMENT NO. 03: CONDITIONS AND LOOPS

Experiment Title: Conditions and Loops

Aim: Choose between cases with if, else if and else, and repeat work with a for loop and a while loop.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: The if, else if and else statements execute code based on conditions. A for loop is useful when the repetition count is known. A while loop repeats as long as its condition remains true.

Algorithm / Procedure:

- Task 1 – Display a grade using conditions.
- Task 2 – Print numbers from 1 to 5 using a for loop.
- Task 3 – Add numbers with a while loop until total exceeds 20.

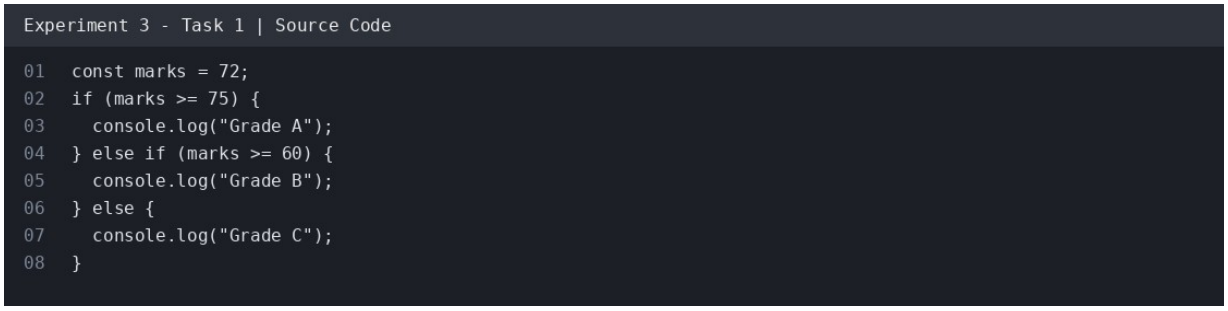
Program / Source Code & Output:

Task 1 – Display a grade using conditions

Source Code:

```
const marks = 72;
if (marks >= 75) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Print numbers from 1 to 5 using a for loop

Source Code:

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Source Code Screenshot:

Experiment 3 - Task 2 | Source Code

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
1
2
3
4
5
```

Task 3 – Add numbers with a while loop until total exceeds 20

Source Code:

```
let total = 0;
let number = 1;
while (total <= 20) {
  total += number;
  number++;
}
console.log(total);
```

Source Code Screenshot:

Experiment 3 - Task 3 | Source Code

```
let total = 0;
let number = 1;
while (total <= 20) {
  total += number;
  number++;
}
console.log(total);
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
21
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Conditions and Loops” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What does if do? — It executes a block when its condition is true.
- When is a for loop useful? — When the number of repetitions is known.
- What does a while loop test? — It tests its condition before each iteration.

EXPERIMENT NO. 04: WRITING FUNCTIONS

Experiment Title: Writing Functions

Aim: Write a function with a return value, an arrow function, and a default parameter.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: A function is a reusable block of code. Parameters receive input values and return sends a result to the caller. Arrow functions provide a shorter function syntax. Default parameters supply a value when an argument is omitted.

Algorithm / Procedure:

- Task 1 – Create and call greet().
- Task 2 – Create an arrow function square.
- Task 3 – Use a default parameter in billWithTax().

Program / Source Code & Output:

Task 1 – Create and call greet()

Source Code:

```
function greet(name) {
  return "Hello, " + name;
}
console.log(greet("Asha"));
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Create an arrow function square

Source Code:

```
const square = number => number * number;
console.log(square(6));
```

Source Code Screenshot:



Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
36
```

Task 3 – Use a default parameter in billWithTax()

Source Code:

```
function billWithTax(amount, taxRate = 0.18) {
  return amount + amount * taxRate;
}
console.log(billWithTax(100));
```

Source Code Screenshot:

```
Experiment 4 - Task 3 | Source Code
01  function billWithTax(amount, taxRate = 0.18) {
02    return amount + amount * taxRate;
03  }
04  console.log(billWithTax(100));
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
118
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Writing Functions” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is a function? — A reusable block of code that performs a task.
- What does return do? — It sends a value back to the caller.
- What is a default parameter? — A parameter value used when no argument is supplied.

EXPERIMENT NO. 05: ARRAYS, ARRAY METHODS AND NESTED ARRAYS

Experiment Title: Arrays, Array Methods and Nested Arrays

Aim: Create an array, read its items, use forEach and filter, and read a value from an array inside an array.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: An array stores an ordered collection of values. Array indexes start at zero and length gives the number of elements. forEach() runs a callback for each item, filter() creates a new array containing items that pass a test, and nested arrays store arrays inside another array.

Algorithm / Procedure:

- Task 1 – Create marks array and print its length and first mark.
- Task 2 – Print each subject using forEach().
- Task 3 – Filter marks of 60 or more.
- Task 4 – Read row 2, column 3 from a nested array.

Program / Source Code & Output:

Task 1 – Create marks array and print its length and first mark

Source Code:

```
const marks = [72, 85, 64, 90];
console.log(marks.length);
console.log(marks[0]);
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Print each subject using forEach()

Source Code:

```
const subjects = ["Maths", "Science", "English"];
subjects.forEach(subject => console.log(subject));
```

Source Code Screenshot:



Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
Maths
Science
English
```

Task 3 – Filter marks of 60 or more

Source Code:

```
const marks = [72, 85, 45, 90];
const passed = marks.filter(mark => mark >= 60);
console.log(passed);
```

Source Code Screenshot:

```
Experiment 5 - Task 3 | Source Code
01  const marks = [72, 85, 45, 90];
02  const passed = marks.filter(mark => mark >= 60);
03  console.log(passed);
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
[ 72, 85, 90 ]
```

Task 4 – Read row 2, column 3 from a nested array

Source Code:

```
const grid = [[1, 2, 3], [4, 5, 6]];
console.log(grid[1][2]);
```

Source Code Screenshot:

```
Experiment 5 - Task 4 | Source Code
01  const grid = [[1, 2, 3], [4, 5, 6]];
02  console.log(grid[1][2]);
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
6
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Arrays, Array Methods and Nested Arrays” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is an array? — An ordered collection of values.
- What does forEach() do? — It calls a function once for each array element.
- What does filter() return? — A new array containing elements that pass the test.

EXPERIMENT NO. 06: JSON TEXT

Experiment Title: JSON Text

Aim: Convert an object to JSON text, convert JSON text back to an object, and catch invalid JSON.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: JSON (JavaScript Object Notation) is a text format for storing and exchanging structured data. JSON.stringify() converts a JavaScript value into JSON text. JSON.parse() converts valid JSON text into a JavaScript value. try and catch handle errors such as invalid JSON.

Algorithm / Procedure:

- Task 1 – Convert an object to JSON text.
- Task 2 – Parse JSON text and print city.
- Task 3 – Catch invalid JSON.

Program / Source Code & Output:

Task 1 – Convert an object to JSON text

Source Code:

```
const student = { name: "Rohan", marks: [70, 80] };
const jsonText = JSON.stringify(student);
console.log(jsonText);
```

Source Code Screenshot:

Experiment 6 - Task 1 | Source Code

```
01  const student = { name: "Rohan", marks: [70, 80] };
02  const jsonText = JSON.stringify(student);
03  console.log(jsonText);
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
{"name":"Rohan","marks":[70,80]}
```

Task 2 – Parse JSON text and print city

Source Code:

```
const text = '{"city":"Pune","pin":411001}';
const data = JSON.parse(text);
console.log(data.city);
```

Source Code Screenshot:

Experiment 6 - Task 2 | Source Code

```
01  const text = '{"city":"Pune","pin":411001}';
02  const data = JSON.parse(text);
03  console.log(data.city);
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
Pune
```

Task 3 – Catch invalid JSON

Source Code:

```
const text = "{name: Rohan}";
try {
  JSON.parse(text);
} catch (error) {
  console.log("Invalid JSON");
}
```

Source Code Screenshot:

```
Experiment 6 - Task 3 | Source Code
01  const text = "{name: Rohan}";
02  try {
03    JSON.parse(text);
04  } catch (error) {
05    console.log("Invalid JSON");
06  }
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
Invalid JSON
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “JSON Text” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is JSON? — A text format for structured data.
- What does JSON.stringify() do? — It converts a JavaScript value to JSON text.
- Why use try and catch? — To handle errors without stopping the program unexpectedly.

EXPERIMENT NO. 07: CONSTRUCTOR FUNCTIONS AND CLASSES

Experiment Title: Constructor Functions and Classes

Aim: Create objects with a constructor function and with an ES6 class, and add a method through the prototype.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: Constructor functions initialize objects when used with new. ES6 classes provide a clearer syntax for creating objects and methods. A prototype allows methods to be shared by objects created from the same constructor.

Algorithm / Procedure:

- Task 1 – Create a Student using a constructor function.
- Task 2 – Create a Book class with a summary method.
- Task 3 – Add result() through Student.prototype.

Program / Source Code & Output:

Task 1 – Create a Student using a constructor function

Source Code:

```
function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}
const student = new Student("Asha", 88);
console.log(student.name);
console.log(student.marks);
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Create a Book class with a summary method

Source Code:

```
class Book {
  constructor(title, pages) {
    this.title = title;
    this.pages = pages;
  }
  summary() {
    return this.title + " has " + this.pages + " pages";
  }
}
```

```
const book = new Book("Algorithms", 320);
console.log(book.summary());
```

Source Code Screenshot:

Experiment 7 - Task 2 | Source Code

```
01  class Book {
02    constructor(title, pages) {
03      this.title = title;
04      this.pages = pages;
05    }
06    summary() {
07      return this.title + " has " + this.pages + " pages";
08    }
09  }
10  const book = new Book("Algorithms", 320);
11  console.log(book.summary());
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Algorithms has 320 pages
```

Task 3 – Add result() through Student.prototype

Source Code:

```
function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}
Student.prototype.result = function () {
  return this.marks >= 40 ? "Pass" : "Fail";
};
const student = new Student("Asha", 88);
console.log(student.result());
```

Source Code Screenshot:

Experiment 7 - Task 3 | Source Code

```
01  function Student(name, marks) {
02    this.name = name;
03    this.marks = marks;
04  }
05  Student.prototype.result = function () {
06    return this.marks >= 40 ? "Pass" : "Fail";
07  };
08  const student = new Student("Asha", 88);
09  console.log(student.result());
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Pass
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Constructor Functions and Classes” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What does new do? — It creates an object from a constructor.
- What is a class? — A template for creating objects.
- What is a prototype? — An object from which JavaScript objects can inherit properties and methods.

EXPERIMENT NO. 08: INHERITANCE, METHODS AND COMPOSITION

Experiment Title: Inheritance, Methods and Composition

Aim: Use extends to make a child class, write a method that checks a value, and place one object inside another.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: Inheritance lets a class reuse and extend another class using extends and super(). A method is a function associated with an object or class. Composition builds a larger object by storing another object inside it.

Algorithm / Procedure:

- Task 1 – Create Person and Teacher classes.
- Task 2 – Check a locker PIN.
- Task 3 – Use composition with Engine and Car.

Program / Source Code & Output:

Task 1 – Create Person and Teacher classes

Source Code:

```
class Person {
  constructor(name) { this.name = name; }
  greet() { return "Hi, I am " + this.name; }
}
class Teacher extends Person {
  constructor(name, subject) {
    super(name);
    this.subject = subject;
  }
}
const teacher = new Teacher("Anita", "Maths");
console.log(teacher.greet());
```

Source Code Screenshot:

Experiment 8 - Task 1 | Source Code

```
01  class Person {
02    constructor(name) { this.name = name; }
03    greet() { return "Hi, I am " + this.name; }
04  }
05  class Teacher extends Person {
06    constructor(name, subject) {
07      super(name);
08      this.subject = subject;
09    }
10  }
11  const teacher = new Teacher("Anita", "Maths");
12  console.log(teacher.greet());
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Hi, I am Anita
```

Task 2 – Check a locker PIN

Source Code:

```
class Locker {
  constructor(pin) { this.pin = pin; }
  checkPin(number) { return number === this.pin; }
}
const locker = new Locker(1234);
console.log(locker.checkPin(1111));
console.log(locker.checkPin(1234));
```

Source Code Screenshot:

Experiment 8 - Task 2 | Source Code

```
01  class Locker {
02    constructor(pin) { this.pin = pin; }
03    checkPin(number) { return number === this.pin; }
04  }
05  const locker = new Locker(1234);
06  console.log(locker.checkPin(1111));
07  console.log(locker.checkPin(1234));
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
false
true
```

Task 3 – Use composition with Engine and Car

Source Code:

```
class Engine {
  constructor(power) { this.power = power; }
}
class Car {
  constructor(power) { this.engine = new Engine(power); }
  start() { return "Car starts with " + this.engine.power + " HP"; }
}
const car = new Car(150);
console.log(car.start());
```

Source Code Screenshot:

Experiment 8 - Task 3 | Source Code

```
01  class Engine {
02    constructor(power) { this.power = power; }
03  }
04  class Car {
05    constructor(power) { this.engine = new Engine(power); }
06    start() { return "Car starts with " + this.engine.power + " HP"; }
07  }
08  const car = new Car(150);
09  console.log(car.start());
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Car starts with 150 HP
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Inheritance, Methods and Composition” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is inheritance? — A class deriving properties and methods from another class.
- What does extends do? — It creates a child class.
- What is composition? — Building an object by including other objects as its parts.

EXPERIMENT NO. 09: DOM, VALIDATION, STORAGE AND QUIZ

Experiment Title: DOM, Validation, Storage and Quiz

Aim: Change a page with a click, check form values in code, save text in local storage, and build a small quiz.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: The Document Object Model (DOM) represents an HTML page as objects that JavaScript can access and change. Event listeners respond to user actions. localStorage stores string data in the browser. Form validation checks input before accepting it.

Algorithm / Procedure:

- Task 1 – Change a heading when a button is clicked.
- Task 2 – Validate a name.
- Task 3 – Save and restore text with localStorage.
- Task 4 – Build a three-option quiz.

Program / Source Code & Output:

Task 1 – Change a heading when a button is clicked

Source Code:

```
<!DOCTYPE html>
<html><body>
<h2 id="heading">Welcome</h2>
<button id="change">Change text</button>
<script>
document.getElementById("change").addEventListener("click", () => {
  document.getElementById("heading").textContent = "Button clicked";
});
</script>
</body></html>
```

Source Code Screenshot:

Experiment 9 - Task 1 | Source Code

```
01  <!DOCTYPE html>
02  <html><body>
03  <h2 id="heading">Welcome</h2>
04  <button id="change">Change text</button>
05  <script>
06  document.getElementById("change").addEventListener("click", () => {
07    document.getElementById("heading").textContent = "Button clicked";
08  });
09  </script>
10  </body></html>
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Before click: Welcome
After click: Button clicked
```

Task 2 – Validate a name

Source Code:

```
function validateForm(name) {
  const errors = [];
  if (name.trim() === "") errors.push("Name is required");
  return errors;
}
console.log(validateForm("Asha"));
console.log(validateForm(""));
```

Source Code Screenshot:

```
Experiment 9 - Task 2 | Source Code

01  function validateForm(name) {
02    const errors = [];
03    if (name.trim() === "") errors.push("Name is required");
04    return errors;
05  }
06  console.log(validateForm("Asha"));
07  console.log(validateForm(""));
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT

[]
[ 'Name is required' ]
```

Task 3 – Save and restore text with localStorage

Source Code:

```
<!DOCTYPE html>
<html><body>
<input id="note" type="text">
<button id="save">Save</button>
<script>
const box = document.getElementById("note");
box.value = localStorage.getItem("note") || "";
document.getElementById("save").addEventListener("click", () => {
  localStorage.setItem("note", box.value);
});
</script>
</body></html>
```

Source Code Screenshot:

```
Experiment 9 - Task 3 | Source Code

01  <!DOCTYPE html>
02  <html><body>
03  <input id="note" type="text">
04  <button id="save">Save</button>
05  <script>
06  const box = document.getElementById("note");
07  box.value = localStorage.getItem("note") || "";
08  document.getElementById("save").addEventListener("click", () => {
09    localStorage.setItem("note", box.value);
10  });
11  </script>
12  </body></html>
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
Browser page: saved note is restored in the text box on reload.
```

Task 4 – Build a three-option quiz

Source Code:

```
<!DOCTYPE html>
<html><body>
<p>What is 2 + 3?</p>
<label><input type="radio" name="answer" value="4">4</label>
<label><input type="radio" name="answer" value="5">5</label>
<label><input type="radio" name="answer" value="6">6</label>
<button id="check">Check answer</button>
<p id="result"></p>
<script>
document.getElementById("check").addEventListener("click", () => {
  const choice = document.querySelector('input[name="answer"]:checked');
  document.getElementById("result").textContent =
    choice && choice.value === "5" ? "Correct" : "Try again";
});
</script>
</body></html>
```

Source Code Screenshot:

Experiment 9 - Task 4 | Source Code

```
01  <!DOCTYPE html>
02  <html><body>
03  <p>What is 2 + 3?</p>
04  <label><input type="radio" name="answer" value="4">4</label>
05  <label><input type="radio" name="answer" value="5">5</label>
06  <label><input type="radio" name="answer" value="6">6</label>
07  <button id="check">Check answer</button>
08  <p id="result"></p>
09  <script>
10  document.getElementById("check").addEventListener("click", () => {
11    const choice = document.querySelector('input[name="answer"]:checked');
12    document.getElementById("result").textContent =
13      choice && choice.value === "5" ? "Correct" : "Try again";
14  });
15  </script>
16  </body></html>
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
Selected 5: Correct
Any other choice: Try again
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “DOM, Validation, Storage and Quiz” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is the DOM? — An object representation of an HTML document.
- What is localStorage? — Browser storage that keeps string values across page reloads.
- What is an event listener? — Code that runs in response to an event.

EXPERIMENT NO. 10: OBJECTS AND DESTRUCTURING

Experiment Title: Objects and Destructuring

Aim: Create an object, read and change its properties, and take values out of an array and an object with destructuring.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: Objects store related data as key-value properties. Properties can be read and updated using dot notation. Destructuring is syntax for unpacking values from arrays or properties from objects into variables.

Algorithm / Procedure:

- Task 1 – Read and update object properties.
- Task 2 – Use array destructuring.
- Task 3 – Use object destructuring.

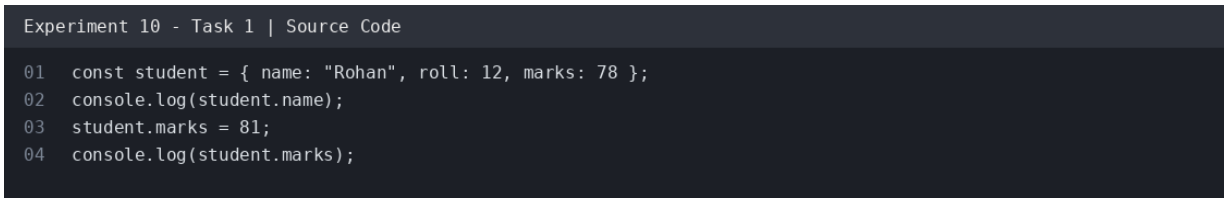
Program / Source Code & Output:

Task 1 – Read and update object properties

Source Code:

```
const student = { name: "Rohan", roll: 12, marks: 78 };
console.log(student.name);
student.marks = 81;
console.log(student.marks);
```

Source Code Screenshot:



Output Screenshot / Output Preview:

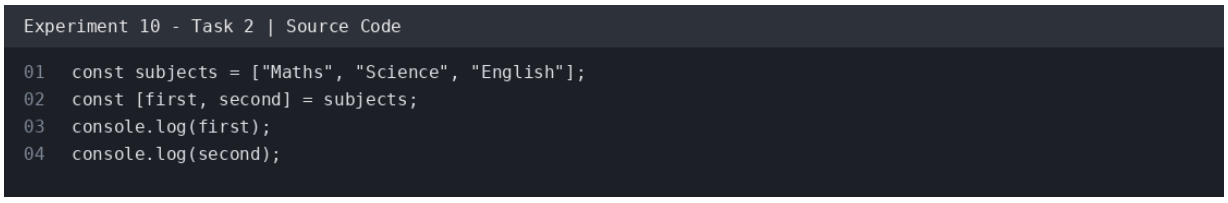


Task 2 – Use array destructuring

Source Code:

```
const subjects = ["Maths", "Science", "English"];
const [first, second] = subjects;
console.log(first);
console.log(second);
```

Source Code Screenshot:



Output Screenshot / Output Preview:

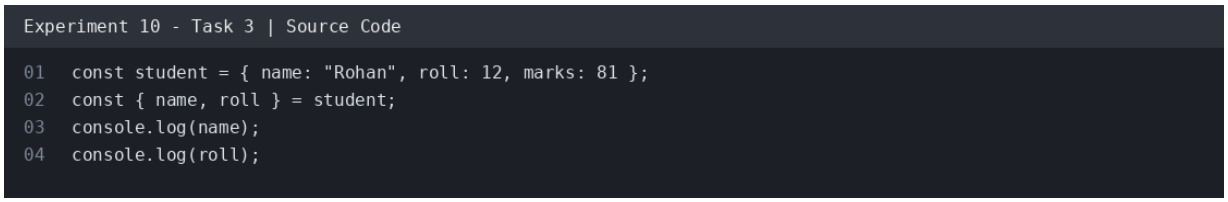


Task 3 – Use object destructuring

Source Code:

```
const student = { name: "Rohan", roll: 12, marks: 81 };
const { name, roll } = student;
console.log(name);
console.log(roll);
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Objects and Destructuring” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is an object? — A collection of properties stored as key-value pairs.
- What is destructuring? — A concise way to unpack values from arrays or objects.
- How do you update a property? — Assign a new value using dot or bracket notation.

EXPERIMENT NO. 11: SCOPE AND CLOSURES

Experiment Title: Scope and Closures

Aim: Explain block scope and global scope, and use a function that returns a function that keeps a value between calls.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: Scope determines where a variable can be accessed. let and const are block-scoped. A global variable can be accessed by functions in its scope. A closure occurs when an inner function remembers variables from its outer function even after the outer function has finished.

Algorithm / Procedure:

- Task 1 – Demonstrate block scope.
- Task 2 – Create a counter using a closure.
- Task 3 – Access a global variable in a function.

Program / Source Code & Output:

Task 1 – Demonstrate block scope

Source Code:

```
let level = "outer";
{
  let level = "inner";
  console.log(level);
}
console.log(level);
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Create a counter using a closure

Source Code:

```
function makeCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const counter = makeCounter();
console.log(counter());
console.log(counter());
```

Source Code Screenshot:

Experiment 11 - Task 2 | Source Code

```
01  function makeCounter() {
02    let count = 0;
03    return function () {
04      count++;
05      return count;
06    };
07  }
08  const counter = makeCounter();
09  console.log(counter());
10  console.log(counter());
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
1
2
```

Task 3 – Access a global variable in a function

Source Code:

```
const school = "Sample College";
function showSchool() {
  console.log("Welcome to " + school);
}
showSchool();
```

Source Code Screenshot:

Experiment 11 - Task 3 | Source Code

```
01  const school = "Sample College";
02  function showSchool() {
03    console.log("Welcome to " + school);
04  }
05  showSchool();
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Welcome to Sample College
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Scope and Closures” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is scope? — The region where a variable can be accessed.
- What is block scope? — A variable is accessible only inside its enclosing block.
- What is a closure? — A function together with the lexical environment it remembers.

EXPERIMENT NO. 12: CALLBACKS AND HIGHER-ORDER FUNCTIONS

Experiment Title: Callbacks and Higher-Order Functions

Aim: Pass a function as an argument to another function and call it.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: A callback is a function passed to another function as an argument. A higher-order function accepts a function, returns a function, or both. Arrow functions are often used for short callback expressions.

Algorithm / Procedure:

- Task 1 – Apply a function twice.
- Task 2 – Format marks with a callback.
- Task 3 – Repeat an action three times.

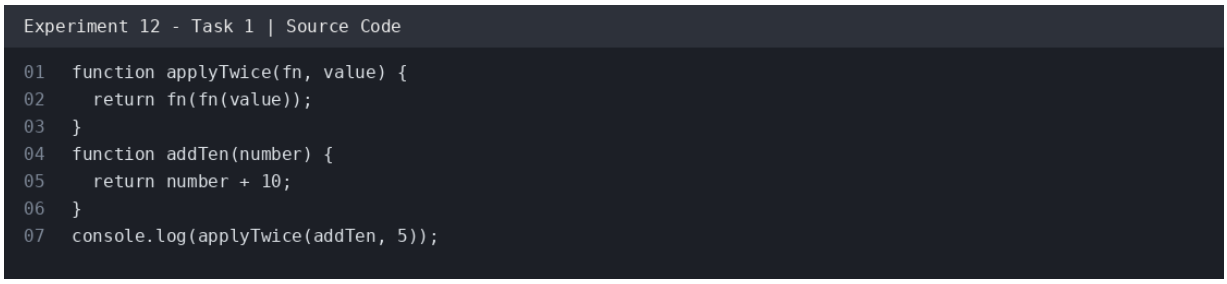
Program / Source Code & Output:

Task 1 – Apply a function twice

Source Code:

```
function applyTwice(fn, value) {
  return fn(fn(value));
}
function addTen(number) {
  return number + 10;
}
console.log(applyTwice(addTen, 5));
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Task 2 – Format marks with a callback

Source Code:

```
function formatMarks(marks, callback) {
  return callback(marks);
}
console.log(formatMarks(76, marks => "Marks: " + marks + "/100"));
```

Source Code Screenshot:

Experiment 12 - Task 2 | Source Code

```
01  function formatMarks(marks, callback) {
02    return callback(marks);
03  }
04  console.log(formatMarks(76, marks => "Marks: " + marks + "/100"));
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Marks: 76/100
```

Task 3 – Repeat an action three times

Source Code:

```
function repeatAction(times, action) {
  for (let round = 1; round <= times; round++) {
    action(round);
  }
}
repeatAction(3, round => console.log("Round " + round));
```

Source Code Screenshot:

Experiment 12 - Task 3 | Source Code

```
01  function repeatAction(times, action) {
02    for (let round = 1; round <= times; round++) {
03      action(round);
04    }
05  }
06  repeatAction(3, round => console.log("Round " + round));
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Round 1
Round 2
Round 3
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Callbacks and Higher-Order Functions” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is a callback? — A function passed as an argument to another function.
- What is a higher-order function? — A function that accepts or returns another function.
- What does applyTwice do here? — It applies the supplied function two times in succession.

EXPERIMENT NO. 13: REDUCE, SOME, EVERY AND MAP

Experiment Title: Reduce, Some, Every and Map

Aim: Use reduce, some, every and map on arrays, and keep the original array unchanged.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: reduce() combines array elements into one accumulated value. some() tests whether at least one element passes a condition. every() tests whether all elements pass. map() creates a new array by transforming each element and leaves the original array unchanged.

Algorithm / Procedure:

- Task 1 – Find the total using reduce().
- Task 2 – Test marks with some() and every().
- Task 3 – Increase each mark using map().

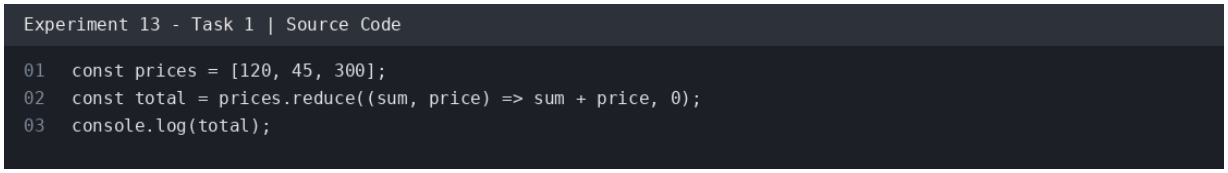
Program / Source Code & Output:

Task 1 – Find the total using reduce()

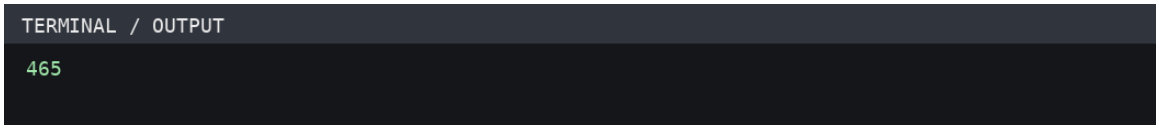
Source Code:

```
const prices = [120, 45, 300];
const total = prices.reduce((sum, price) => sum + price, 0);
console.log(total);
```

Source Code Screenshot:



Output Screenshot / Output Preview:

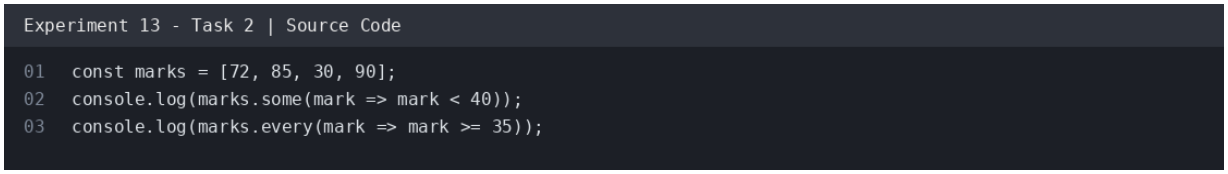


Task 2 – Test marks with some() and every()

Source Code:

```
const marks = [72, 85, 30, 90];
console.log(marks.some(mark => mark < 40));
console.log(marks.every(mark => mark >= 35));
```

Source Code Screenshot:



Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
true
false
```

Task 3 – Increase each mark using map()

Source Code:

```
const marks = [72, 85, 64, 90];
const increased = marks.map(mark => mark + 5);
console.log(increased);
console.log(marks);
```

Source Code Screenshot:

```
Experiment 13 - Task 3 | Source Code
01  const marks = [72, 85, 64, 90];
02  const increased = marks.map(mark => mark + 5);
03  console.log(increased);
04  console.log(marks);
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
[ 77, 90, 69, 95 ]
[ 72, 85, 64, 90 ]
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Reduce, Some, Every and Map” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What does reduce() return? — One accumulated value.
- What does some() check? — Whether at least one element satisfies a condition.
- Does map() change the original array? — No, it returns a new array.

EXPERIMENT NO. 14: TIMERS AND INTERVALS

Experiment Title: Timers and Intervals

Aim: Run code after a delay with setTimeout, cancel a timer, and repeat code with setInterval.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: setTimeout() schedules a callback to run once after a delay. clearTimeout() cancels a scheduled timeout. setInterval() repeatedly runs a callback after a fixed delay, and clearInterval() stops the repetition.

Algorithm / Procedure:

- Task 1 – Observe setTimeout() with zero delay.
- Task 2 – Cancel a timeout.
- Task 3 – Print three ticks using setInterval().

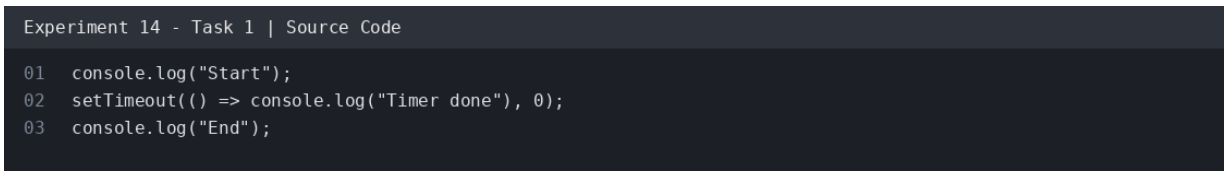
Program / Source Code & Output:

Task 1 – Observe setTimeout() with zero delay

Source Code:

```
console.log("Start");
setTimeout(() => console.log("Timer done"), 0);
console.log("End");
```

Source Code Screenshot:



Output Screenshot / Output Preview:

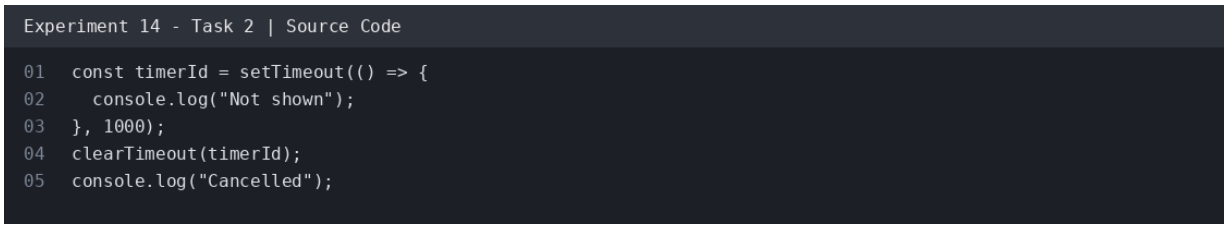


Task 2 – Cancel a timeout

Source Code:

```
const timerId = setTimeout(() => {
  console.log("Not shown");
}, 1000);
clearTimeout(timerId);
console.log("Cancelled");
```

Source Code Screenshot:



Output Screenshot / Output Preview:

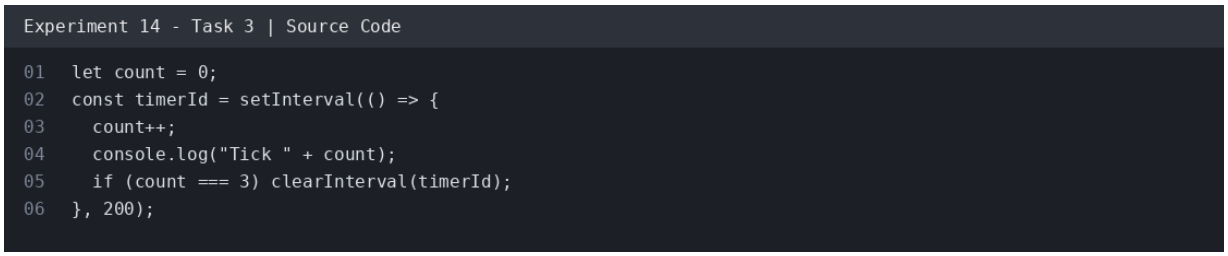


Task 3 – Print three ticks using setInterval()

Source Code:

```
let count = 0;
const timerId = setInterval(() => {
  count++;
  console.log("Tick " + count);
  if (count === 3) clearInterval(timerId);
}, 200);
```

Source Code Screenshot:



Output Screenshot / Output Preview:



Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Timers and Intervals” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What does setTimeout() do? — It schedules a callback to run once after a delay.
- How do you cancel a timeout? — With clearTimeout() and its timer ID.
- What does clearInterval() do? — It stops a repeating interval.

EXPERIMENT NO. 15: PROMISES AND ASYNC/AWAIT WITH LOCAL DATA

Experiment Title: Promises and async/await with Local Data

Aim: Create a Promise that resolves or rejects, use async/await with try and catch, and load local data as a Promise.

Software / Tools Required: JavaScript-enabled web browser, Node.js, VS Code or any text editor, and Command Prompt / Terminal.

Theory: A Promise represents the eventual completion or failure of an asynchronous operation. It can be fulfilled or rejected. async functions return promises, and await pauses within an async function until a promise settles. try and catch handle rejected promises.

Algorithm / Procedure:

- Task 1 – Resolve a Promise for valid marks.
- Task 2 – Reject a Promise for out-of-range marks.
- Task 3 – Handle Promise results with async/await.
- Task 4 – Load local users with a Promise.

Program / Source Code & Output:

Task 1 – Resolve a Promise for valid marks

Source Code:

```
function checkMarks(marks) {
  return new Promise((resolve, reject) => {
    if (marks >= 0 && marks <= 100) resolve(marks);
    else reject(new Error("Marks out of range"));
  });
}
checkMarks(80).then(value => console.log(value));
```

Source Code Screenshot:

Experiment 15 - Task 1 | Source Code

```
01  function checkMarks(marks) {
02    return new Promise((resolve, reject) => {
03      if (marks >= 0 && marks <= 100) resolve(marks);
04      else reject(new Error("Marks out of range"));
05    });
06  }
07  checkMarks(80).then(value => console.log(value));
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
80
```

Task 2 – Reject a Promise for out-of-range marks

Source Code:

```
function checkMarks(marks) {
  return new Promise((resolve, reject) => {
    if (marks >= 0 && marks <= 100) resolve(marks);
    else reject(new Error("Marks out of range"));
  });
}
checkMarks(120).catch(error => console.log(error.message));
```

Source Code Screenshot:

Experiment 15 - Task 2 | Source Code

```
01  function checkMarks(marks) {
02    return new Promise((resolve, reject) => {
03      if (marks >= 0 && marks <= 100) resolve(marks);
04      else reject(new Error("Marks out of range"));
05    });
06  }
07  checkMarks(120).catch(error => console.log(error.message));
```

Output Screenshot / Output Preview:

TERMINAL / OUTPUT

```
Marks out of range
```

Task 3 – Handle Promise results with async/await

Source Code:

```
function checkMarks(marks) {
  return new Promise((resolve, reject) => {
    if (marks >= 0 && marks <= 100) resolve(marks);
    else reject(new Error("Marks out of range"));
  });
}
async function run() {
  try {
    console.log(await checkMarks(65));
    await checkMarks(-5);
  } catch (error) {
    console.log("Caught: " + error.message);
  }
}
run();
```

Source Code Screenshot:

Experiment 15 - Task 3 | Source Code

```
01  function checkMarks(marks) {
02    return new Promise((resolve, reject) => {
03      if (marks >= 0 && marks <= 100) resolve(marks);
04      else reject(new Error("Marks out of range"));
05    });
06  }
07  async function run() {
08    try {
09      console.log(await checkMarks(65));
10      await checkMarks(-5);
11    } catch (error) {
12      console.log("Caught: " + error.message);
13    }
14  }
15  run();
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
65
Caught: Marks out of range
```

Task 4 – Load local users with a Promise

Source Code:

```
function getUsers() {
  return new Promise(resolve => {
    setTimeout(() => resolve([
      { id: 1, name: "Asha" },
      { id: 2, name: "Rohan" },
      { id: 3, name: "Meera" }
    ]), 100);
  });
}
async function showUsers() {
  const users = await getUsers();
  users.forEach(user => console.log(user.name));
}
showUsers();
```

Source Code Screenshot:

```
Experiment 15 - Task 4 | Source Code
01  function getUsers() {
02    return new Promise(resolve => {
03      setTimeout(() => resolve([
04        { id: 1, name: "Asha" },
05        { id: 2, name: "Rohan" },
06        { id: 3, name: "Meera" }
07      ]), 100);
08    });
09  }
10  async function showUsers() {
11    const users = await getUsers();
12    users.forEach(user => console.log(user.name));
13  }
14  showUsers();
```

Output Screenshot / Output Preview:

```
TERMINAL / OUTPUT
Asha
Rohan
Meera
```

Input / Test Data: Values and strings are initialized in the program as specified in the JavaScript lab assignment.

Result: Thus, the experiment “Promises and async/await with Local Data” was performed using JavaScript. The programs were written and the expected outputs were obtained.

Viva-Voce Questions with Answers:

- What is a Promise? — An object representing the eventual result of an asynchronous operation.
- What does await do? — It waits for a promise to settle inside an async function.
- How is a rejected promise handled? — With catch() or try/catch in an async function.